

Files

main

Go to file

questao1

questao2

questao2.md

questao2.py

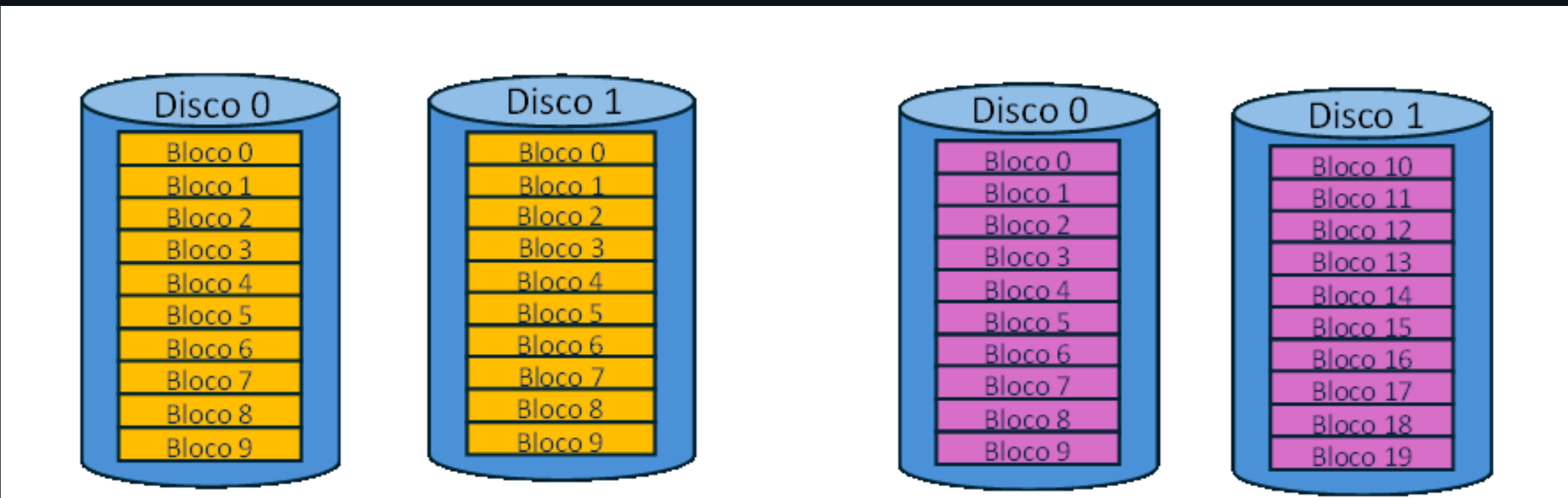
README.md

Questão 2: (60 pontos)

Simulação da operação do RAID4 (e RAID5)

A tecnologia RAID (Redundant Array of Inexpensive Disks) tem sido muito empregada em datacenters como meio de obter discos de grande capacidade a partir de discos convencionais. Existem vários níveis de RAID. No mais simples desses, vários discos são combinados para ao final obter a capacidade agregada como sendo oriunda de um só disco. A implementação da tecnologia é mais comum usando hardware, mas os sistemas operacionais mais populares disponibilizam implementações em software.

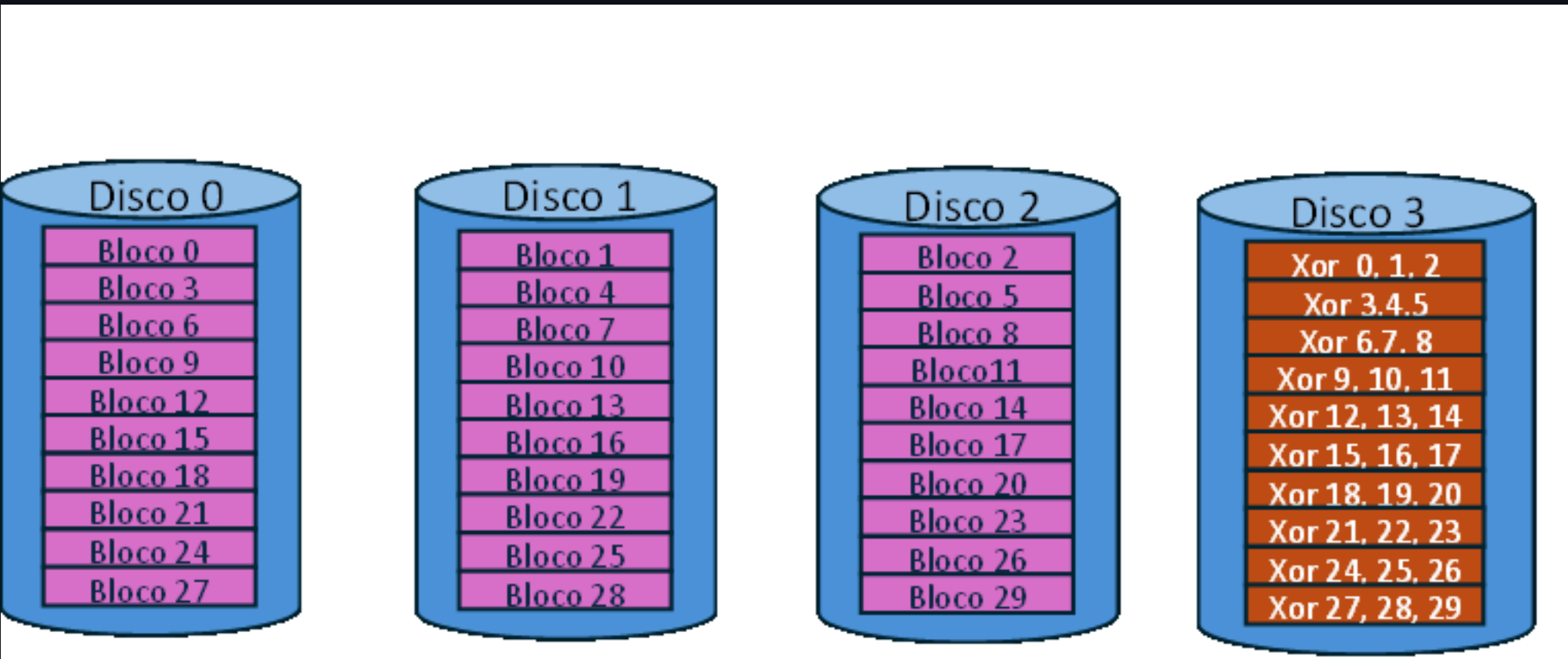
Para entender o RAID0, considere que dois discos de 10 KB estão disponíveis e deseja-se obter um disco de 20 KB. Inicialmente cada disco está dividido em dez blocos de 1 KB. Nesse cenário as organizações físicas e lógicas estão representadas na figura seguinte:



Nesse contexto, os dois discos são considerados como um só disco lógico. Assim sendo, quando for solicitada a leitura (ou escrita) de dados na posição 14750, por exemplo, deve recuperá-los no bloco lógico 14, que em termos físicos, corresponde ao bloco 4 do disco 1.

A tecnologia RAID5 é mais complexa, pois introduz uma lógica que permite recuperar dados em caso de um dos discos do array apresentar problema. Além disso, ao invés de numerar todos os blocos sequencialmente do disco 0 e depois usar o disco 1, eles são utilizados alternadamente.

Antes de compreender todos os detalhes do RAID5, considere o RAID4 que é uma simplificação do RAID5. Na Figura seguinte estão quatro discos em RAID4, já na estrutura lógica.



Existem no RAID4 duas mudanças muito relevantes em relação ao RAID0:

- Os blocos não são alocados completamente no disco 0, para depois alocar no disco 1 e nos demais discos. Eles estão alocados na “horizontal”, ou seja, aloca-se o bloco 0 no disco 0, e um bloco em cada um dos discos subsequentes, até todos os discos tenham um bloco. A seguir o disco 0 recebe um novo bloco e o processo se repete;
- O último disco é denominado de disco de paridade e não armazena blocos de dados, mas blocos que contemplam a operação xor entre os bytes em cada um dos blocos de dados na posição correspondente. Nesse cenário em que cada bloco tem 1KB, o primeiro byte do bloco 0 do disco 3 é um xor dos primeiros bytes nos blocos 0, 1 e 2 (nos discos 0, 1 e 2). O segundo byte do bloco 0 no disco 3 é um xor dos segundos bytes nos blocos 0, 1 e 2 (nos discos 0, 1 e 2). Como um exemplo geral, o byte 20 do bloco 5 do disco 3 é um xor dos bytes nas posições 20 dos blocos 15, 16 e 17 (nos discos 0, 1 e 2).

A estratégia do RAID4 é tal que se um dos discos com dados for perdido, os seus dados ainda podem ser recuperados, mediante a operação de xor entre os discos de dados remanescentes e o disco de paridade. A operação de recuperação é a mesma utilizada para gerar os dados de paridade, só que nesse caso utiliza-se os valores nele armazenados para obter os dados do disco defeituoso.

Com base nessa explicação faça um programa em Python, que implementa as seguintes operações e as apresenta em um menu para o usuário:

- inicializaRAID:** Pergunta ao usuário quantos discos serão utilizados em RAID, o tamanho dos discos (o mesmo para todos) e o tamanho do bloco. Os arquivos devem ser criados em uma pasta que o usuário também deve informar.

Essa função deve criar um arquivo para cada um dos discos (disco0.bin, disco1.bin, disco2.bin, discoX.bin). O arquivo discoX.bin representa o disco de paridade (X é antecessor do número de discos informado pelo usuário). Cada arquivo representando discos de dados deve ter todo o seu conteúdo zerado, enquanto o arquivo que guarda o disco de paridade deve ter os dados calculados pela aplicação do xor dos arquivos de dados;

- obtemRAID:** Essa operação pergunta ao usuário as mesmas informações de InicializaRAID, mas em vez criar os arquivos, busca os arquivos criados anteriormente com inicializaRAID;

- escreveRAID:** Pergunta ao usuário um conjunto de dados a gravar no RAID e a posição onde iniciar a gravação. Essa posição pode ser qualquer valor entre zero e o tamanho lógico do RAID -1. Por exemplo, se o RAID tem cinco discos (quatro de dados e um de paridade) e o tamanho dos discos é 10000 bytes, então a posição pode ser qualquer valor entre 0 e 39999. O programa deve identificar em que arquivo(s) gravar os dados e que posição dentro do(s) arquivo(s).

Após a escrita no arquivo correto, o disco de paridade deve ser atualizado;

- leRAID:** Pergunta ao usuário informações sobre dados a ler do RAID. O usuário informa a posição e quantos bytes ler. A lógica para encontrar o arquivo de onde ler é a mesma da escrita. A paridade não necessita ser atualizada;

- removeDiscoRAID:** O usuário indica um disco a remover do RAID4 (simulando um defeito). O arquivo que representa o disco deve ser apagado pelo programa. Ainda assim, as operações seguintes de leitura e escrita devem operar normalmente, mesmo quando envolvem o disco removido.

Quando a leitura envolve o disco removido, os dados devem ser obtidos mediante xor nos demais discos e no disco de paridade. A operação de escrita no disco removido gera efeitos apenas no disco de paridade, a fim de permitir (indiretamente) que os dados sendo gravados possam ser recuperados em futuras leituras;

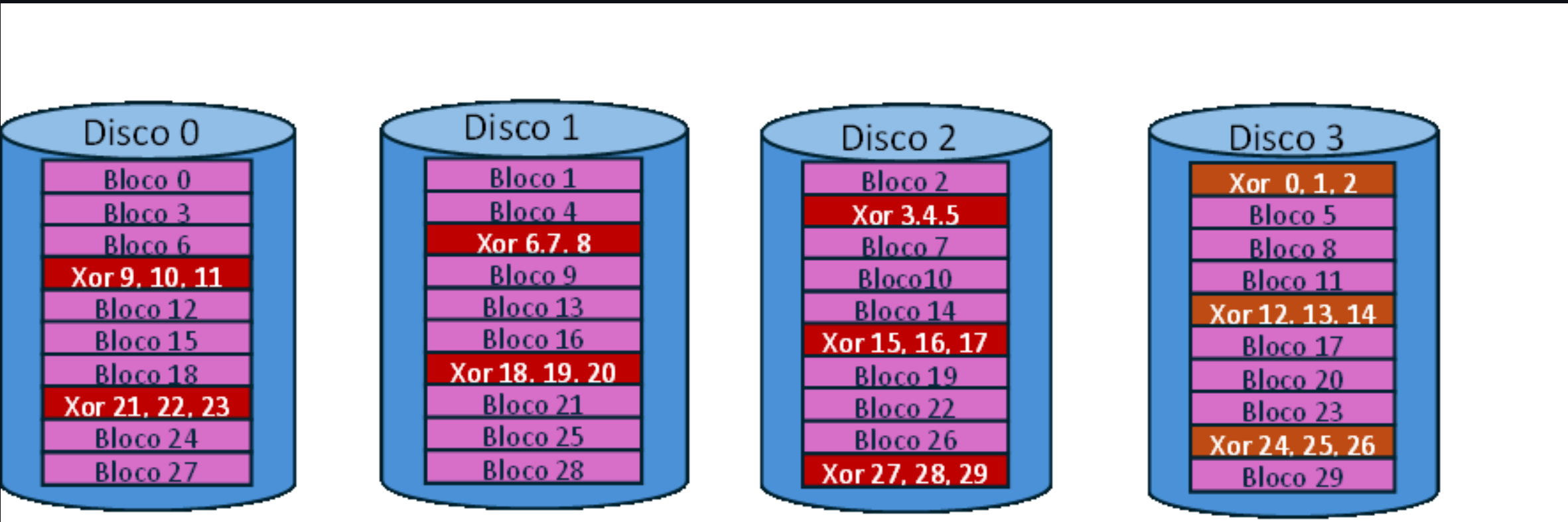
- constroiDiscoRAID:** O usuário pede para reconstruir o disco defeituoso. Um novo arquivo deve ser criado e ter seu conteúdo gerado a partir dos discos remanescentes e o disco de paridade.

Procure desenvolver o programa com uma ou mais funções para implementar cada uma das funcionalidades. No início, considere que as leituras/escritas não ultrapassam limites de bloco. Na segunda versão, implemente completamente.

Algumas dicas:

- As operações nos arquivos envolvem leitura/escrita de dados em formato binário, portanto devem ser abertos no modo “ab+” (leia a documentação da função `open`, ou simplesmente digite `help(open)` no interpretador).
- Use extensivamente a função `seek` para localizar as posições de leitura/escrita nos arquivos que simulam os discos.

O funcionamento do RAID5 é similar ao RAID4, mas a fim de evitar sobrecarga no disco de paridade, haja vista que qualquer escrita em um dos discos de dados implica em escrever o disco de paridade também, a paridade é gravada uniformemente em todos os discos, conforme representado na Figura seguinte:



PONTO BÔNUS (20 pontos): Faça uma nova versão do programa, mas agora usando o RAID5.